

2026 Summer YBIGTA

Git

28기 김아연

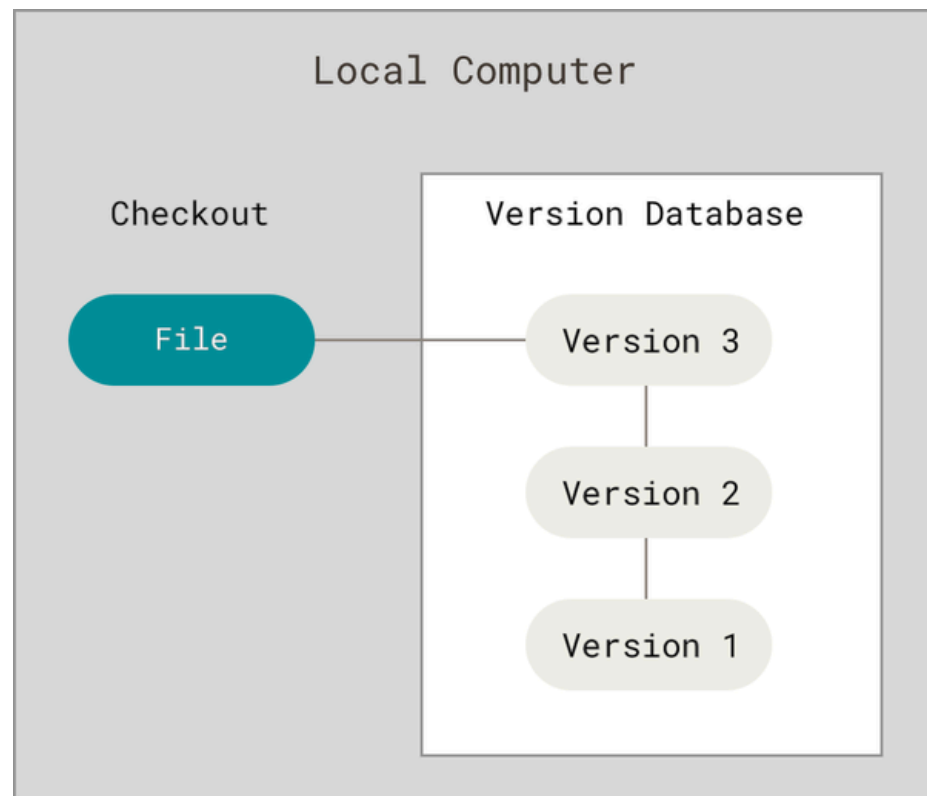
목 차

1. Git 기본 개념
2. Git command
3. 협업

Git 기본개념

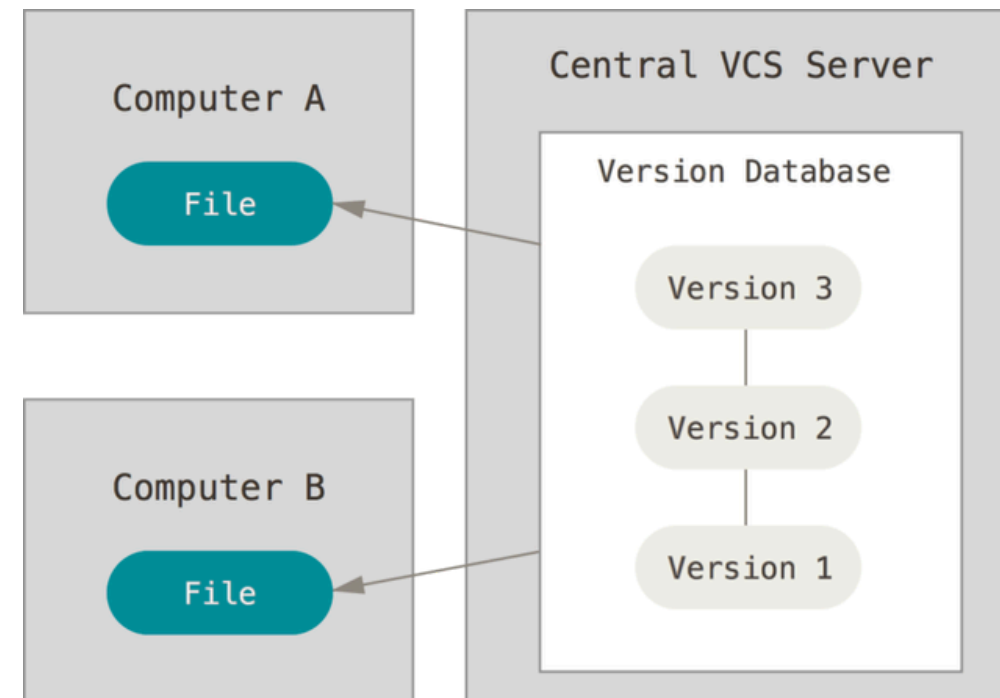
Version Control System

VCS = 코드 변경 이력 관리자



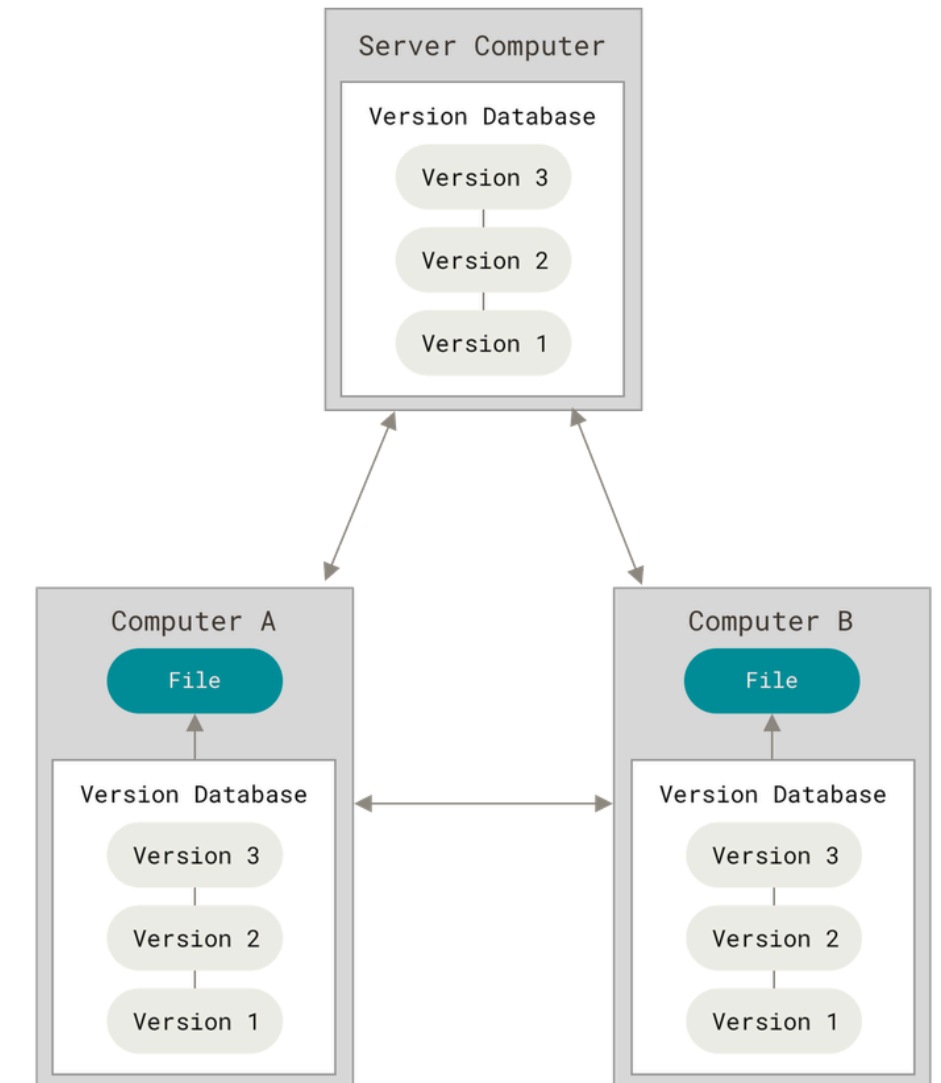
Local VCS
로컬 버전관리

RCS



CVCS
중앙집중식 버전관리

CVS, Subversion, Perforce



DVCS
분산 버전관리 시스템

Git, Mercurial

Git 왜 쓰나요?

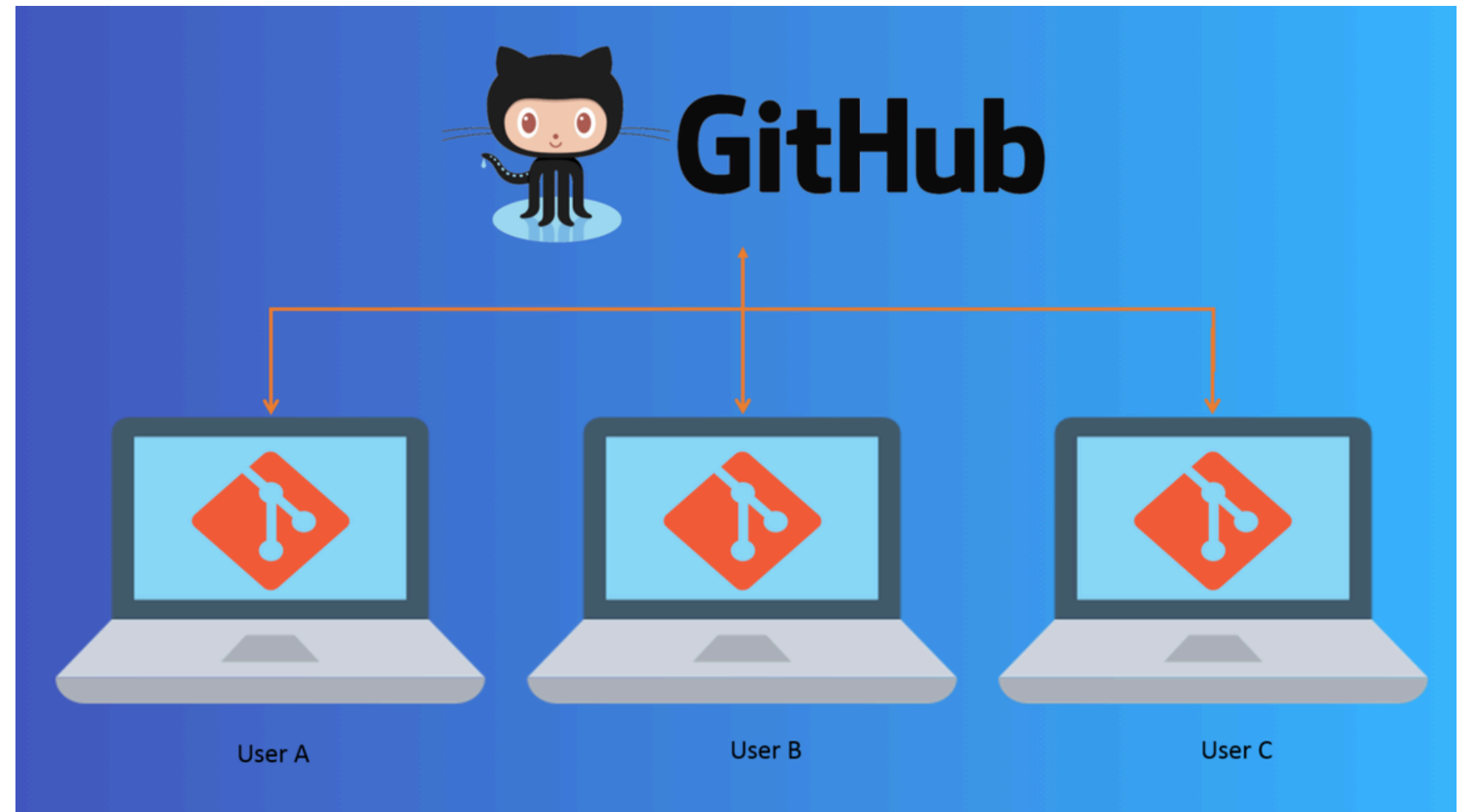
- 파일 변경 내역 보존 및 관리
- 코드 복구
- 안전한 코드 수정 및 업데이트
- 협업



GitHub

Git ≠ GitHub !!

- 로컬 저장소: 내 컴퓨터 안
- 원격 저장소: GitHub 같은 서버
- → **둘을 push/pull 하면서 동기화**



Git 설치 및 설정

- Git을 먼저 설치해줍니다!

 [Git 설치 가이드 페이지](#)

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치하세요! https://aka.ms/PSWindows

개인 및 시스템 프로필을 로드하는 데 2538ms가 걸렸습니다.
(base) PS C:\Users\> git --version
git version 2.42.0.windows.2
(base) PS C:\Users\> |
```

git --version으로 버전 정보 나오면 성공!

- Git 기본 설정을 해줍니다!
 - git config --global user.name "<user_name>"
 - git config --global user.email "<user_email>"
 - git config --global init.defaultBranch main
 - git config --global core.editor "code --wait"

누가 git을 사용하고 있는지 보여주기 위한 아이디 등록

기본 branch 이름을 main으로

기본 에디터를 VScode로 설정

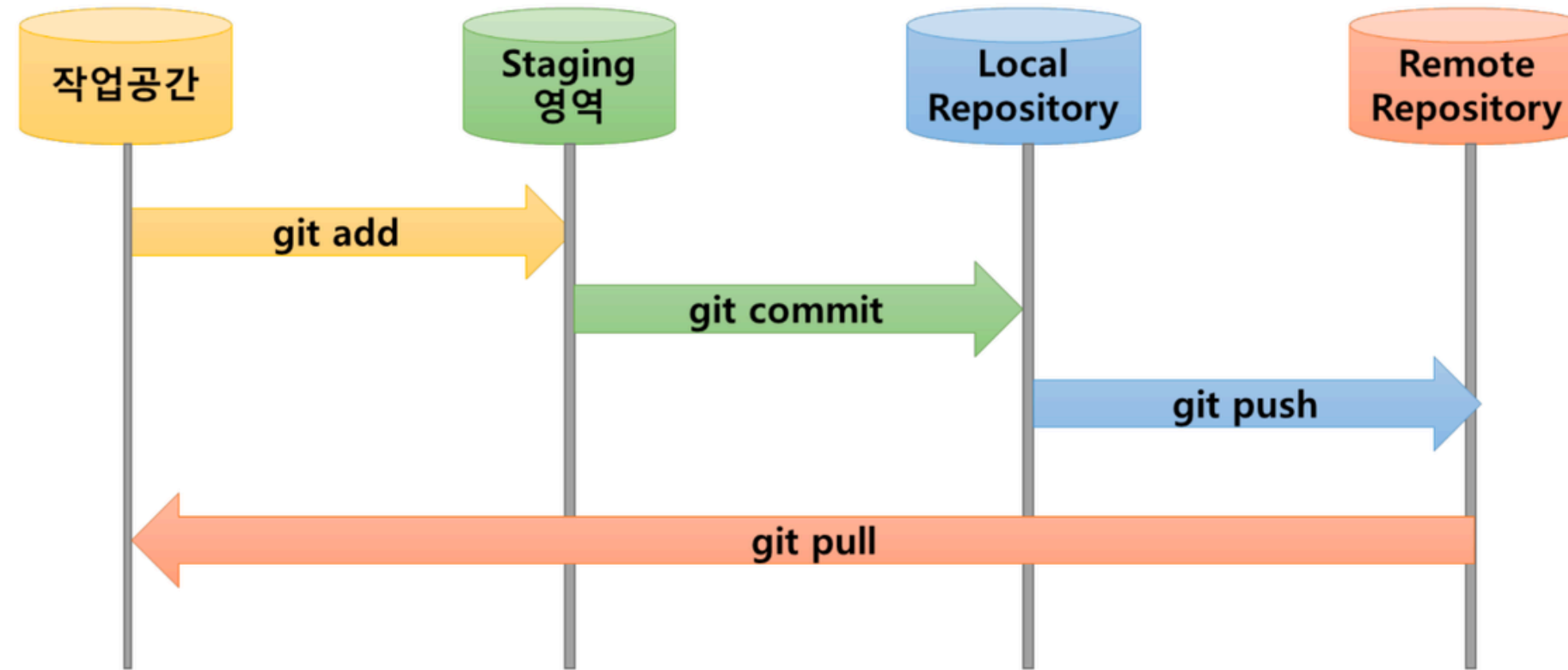
Git 설치 및 설정

- Git 기본 설정을 해줍니다!

- `git config --global diff.tool vscode`
- `git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'`
- `git config --global difftool.prompt false` difftool 기본 에디터를 VScode로 설정
- `git config --global -e` 이외에도 다양한 설정값들 확인 및 수정 가능

(참고링크)_git 최초 설정

Git의 데이터 흐름



- 실제로 코드를 수정하는 곳
- 커밋할 변경사항을 고르는 공간
- 내 컴퓨터에 저장된 Git 커밋 히스토리
- GitHub 같은 공유 서버

Git graph

The first screenshot shows the VS Code Extensions Marketplace with the search term 'git graph'. The 'Git Graph' extension by mhutchie is highlighted with a red box. It has 11.6M downloads and a 5-star rating. Other extensions like 'Git Graph v3', 'Git Graph Revamped', and 'Git History Graph' are also visible.

The second screenshot shows the 'Git Graph' extension installed and active. The main interface displays a list of commits under the 'master' branch. The 'Uncommitted Changes (1)' section shows a file named 'hello_world.txt' with a message 'Web발신'.

The third screenshot shows a context menu for a specific commit (4dc6d314) in the 'master' branch. The menu options are:

- View Diff
- View File at this Revision
- View Diff with Working File
- Open File
- Copy Absolute File Path to Clipboard
- Copy Relative File Path to Clipboard

The commit details shown are:

- Commit:** 4dc6d314d693b696ed42047d2bf0d992e71ad42e
- Parents:** [6b1398eb59612228356488741e2d3b3b4b60464f](#)
- Author:** wognsths <2021122006@yonsei.ac.kr>
- Committer:** wognsths <2021122006@yonsei.ac.kr>
- Date:** Sun Jul 06 2025 00:58:49 GMT+0900 (Korean Standard Time)
- Message:** Revert "Web발신"

Git command

Git 협업 시나리오

1. **A가 main을 만들고 올린 뒤,**
2. **B가 main을 clone해서 새로운 브랜치에서 작업한 후 main에 merge.**

(1) git 시작

로컬 프로젝트 생성 & 깃 초기화

```
● → git-session mkdir proj
● → git-session cd proj
● → proj echo 'print("v1")' > app.py
● → proj git init
    Initialized empty Git repository in /Users/eomjoonseo/qit-session/proj/.git/
```

git init: 로컬 폴더를 Git 저장소로 만들기

내 컴퓨터에 프로젝트 폴더를 만들고 Git이 추적을 시작하도록 설정합니다!

(2) 파일 스테이징 & 커밋

add + commit

```
● → new_project git:(main) ✕ git add app.py
● → new_project git:(main) ✕ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.py
```

```
● → new_project git:(main) ✕ git commit -m "feat: login 기능 추가"

[main c874d65] feat: login 기능 추가
 1 file changed, 1 insertion(+)
● → new_project git:(main) git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

git add <파일> 수정한 파일을 커밋 대상으로 올리기(스테이징)

git status: 지금 작업 상태 파악 (수정/스테이징 여부)

git commit -m "메시지" 스테이징된 변경을 로컬 저장버전으로 남기기

작업한 파일을 스테이징 영역에 올리고 커밋합니다

(3) 저장소 연결 + 최초 push

```
● → proj git:(main) git branch -M main  
● → proj git:(main) git remote add origin https://github.com/Eoo0m/new_project.git  
● → proj git:(main) git push -u origin main  
Enumerating objects: 3, done.  
Counting objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 216 bytes | 216.00 KiB/s, done.  
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0  
To https://github.com/Eoo0m/new_project.git  
* [new branch]      main -> main  
branch 'main' set up to track 'origin/main'.
```

git branch -M main: 기본 브랜치 이름을 main으로 설정

git remote add origin <URL>: 내 Git을 GitHub 원격 저장소와 연결

git push -u origin main: 로컬 main을 GitHub main에 올리고, 이후 기본 push 대상 저장(-u)

내 컴퓨터의 기록을 GitHub 서버와 연결하고 밀어 넣습니다

(4) gitHub에서 프로젝트 가져오기

다른 사람이 만든프로젝트 가져오기 (B 입장)

```
● → Eoom git clone https://github.com/Eoo0m/new_project.git
Cloning into 'new_project'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

git clone <URL>: GitHub 저장소 통째로 다운로드 + 연결까지 자동 설정

이미 시작된 프로젝트를 내 컴퓨터로 그대로 복제해옵니다

(5) 새로운 브랜치 생성 & 작업

login 이라는 브랜치를 파서 작업

```
• → Eoom cd new_project
• → new_project git:(main) git switch -c login
  Switched to a new branch 'login'
• → new_project git:(login) echo 'print("login ready")' >> app.py
• → new_project git:(login) x git add app.py
• → new_project git:(login) x git commit -m "feat: login 기능 추가"

[login 965a30d] feat: login 기능 추가
1 file changed, 1 insertion(+)
• → new_project git:(login) git push origin login
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 285 bytes | 285.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'login' on GitHub by visiting:
remote:   https://github.com/Eoo0m/new_project/pull/new/login
remote:
To https://github.com/Eoo0m/new_project.git
 * [new branch]      login -> login
```

git switch -c login: login 브랜치 새로 만들고 이동 (기능 개발 공간 확보)

안전한 코드 수정을 위해 main이 아닌 새로운 브랜치 생성!

(6) main으로 돌아가서 merge

main으로 이동하여 pull

```
● → new_project git:(login) git switch main  
Switched to branch 'main'  
Your branch is up to date with 'origin/main'.  
● → new_project git:(main) git pull origin main  
From https://github.com/Eoo0m/new_project  
* branch          main          -> FETCH_HEAD  
Already up to date.
```

git switch main: main 브랜치로 이동

git pull origin main: GitHub 최신 main 가져와 로컬 main 업데이트

작업을 끝내고 메인 브랜치로 돌아와 최신 코드를 컴퓨터로 가져옵니다.

(7) merge이후 반영

merge & push

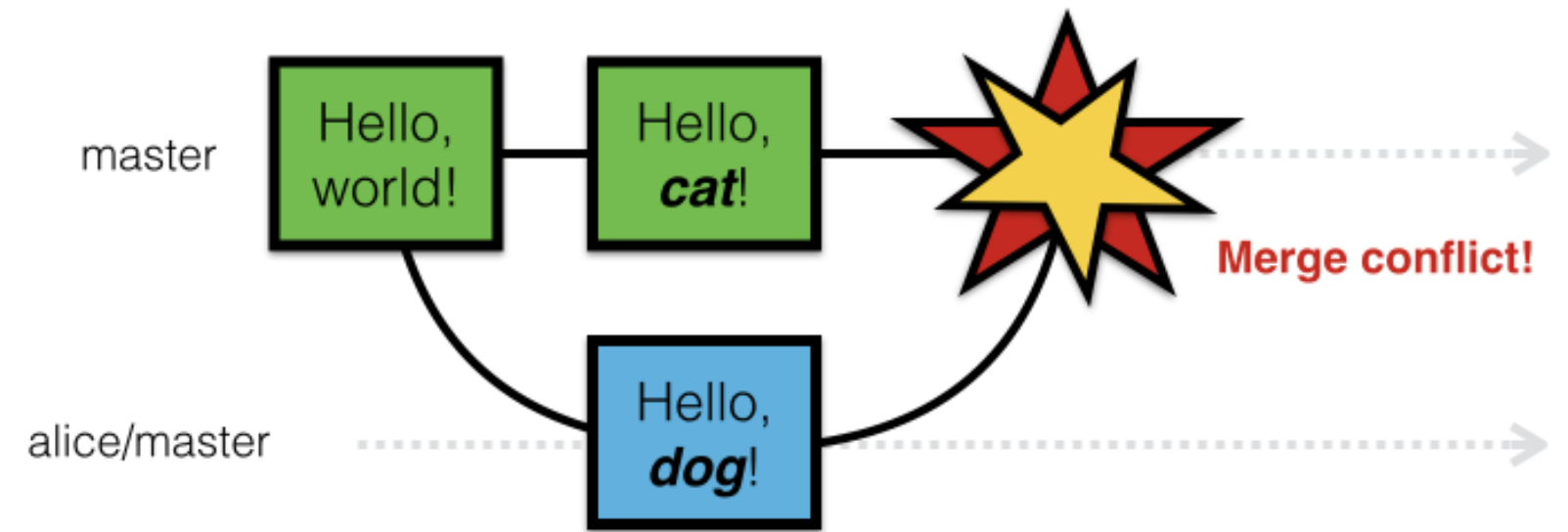
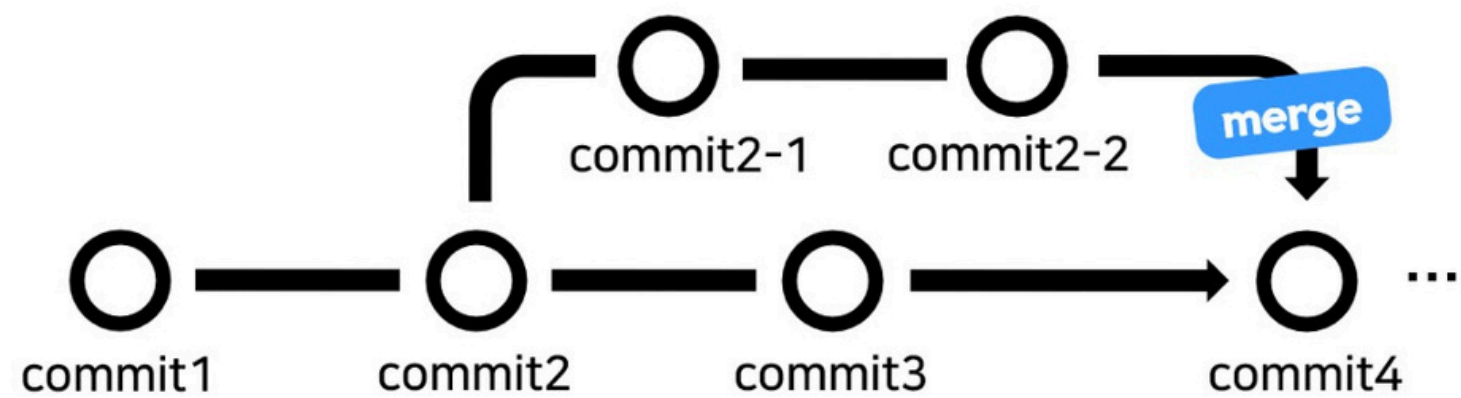
```
● → new_project git:(main) git merge login
Updating aa558b7..965a30d
Fast-forward
 app.py | 1 +
 1 file changed, 1 insertion(+)
● → new_project git:(main) git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Eoo0m/new_project.git
 aa558b7..965a30d  main -> main
```

git merge login: login 내용(main 기준) 병합 → 기능 반영

git push origin main: merge된 main을 GitHub에 올리기

작업 내용을 병합하고합쳐진 코드를 서버에 반영합니다.

git merge & resolve conflict



코드를 보면서 수동으로 수정하고, conflict 해결하고 나서

- git add 파일명
- git commit -m “커밋메시지”

git merge && resolve conflict

Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes

<<<<<< HEAD (Current Change)

Hello from MAIN

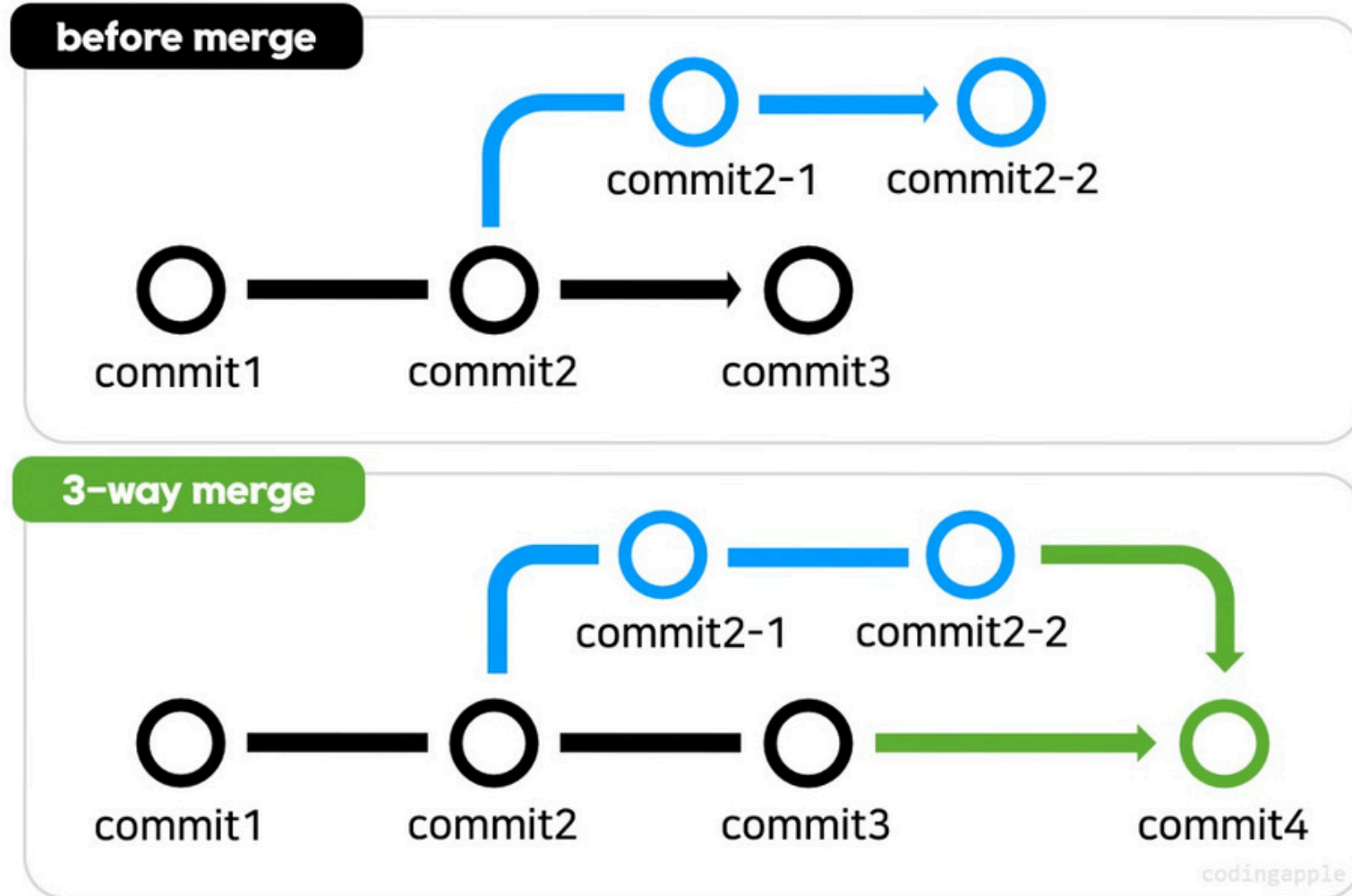
=====
=====

Hello from FEATURE

>>>>>> feature/conflict (Incoming Change)

- Current Change (HEAD) → 지금 브랜치(main)의 내용
- incoming Change → 합쳐오려는 브랜치(feature/conflict)의 내용

git merge +



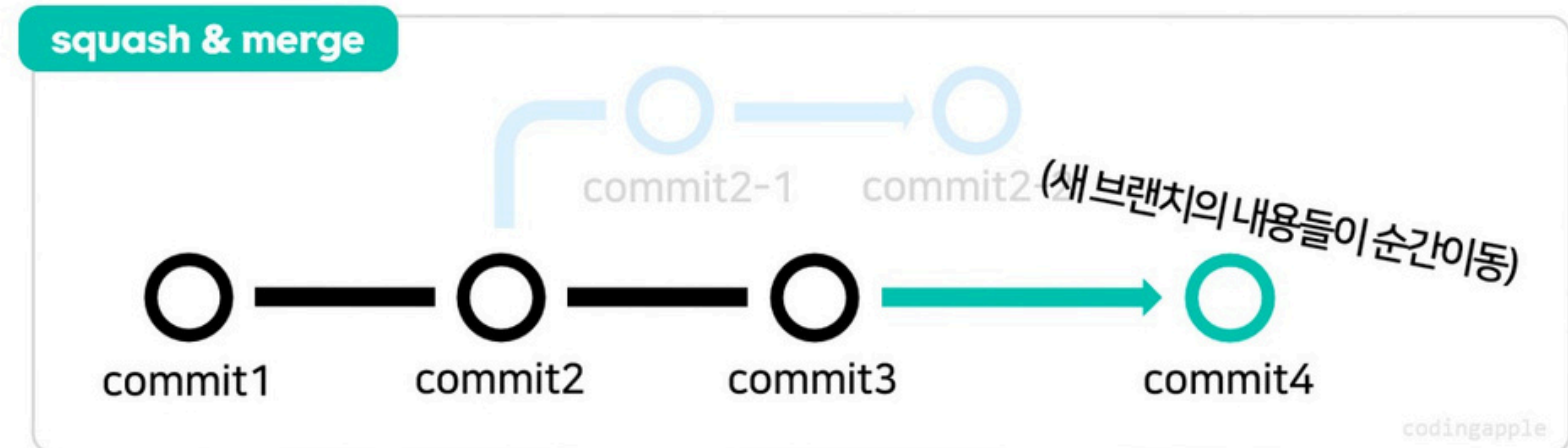
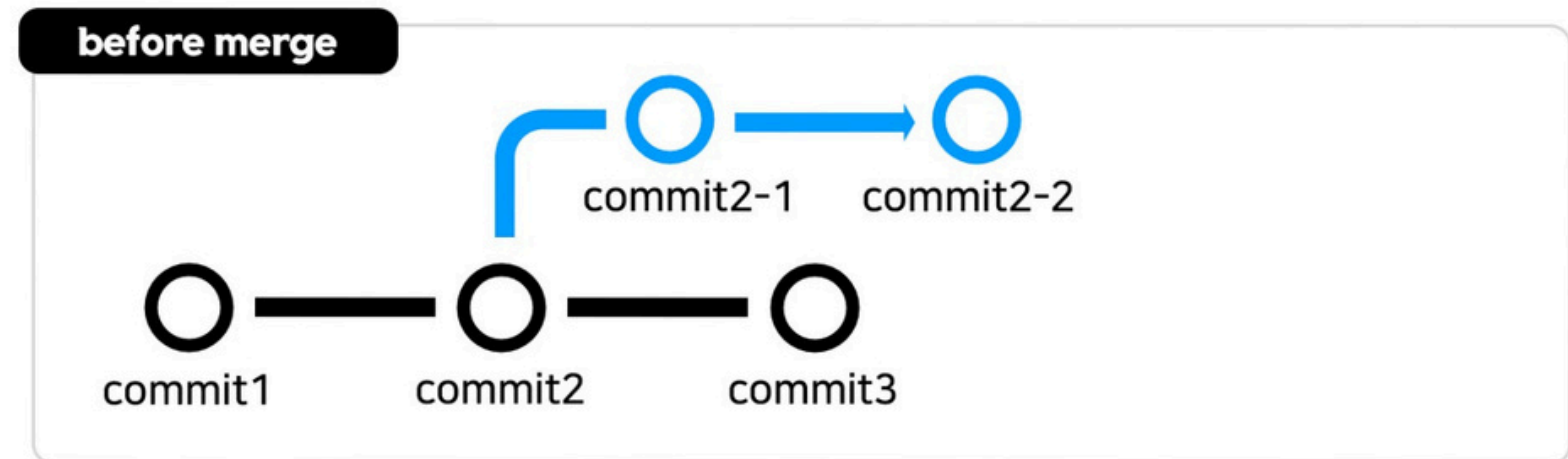
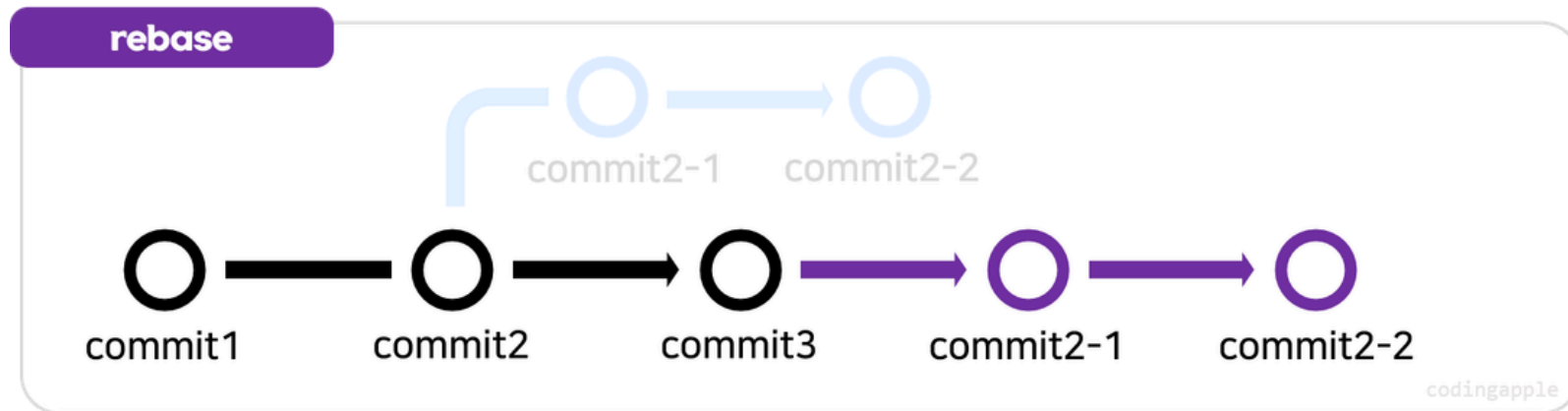
3-way merge

1. 공통 조상 참고 (main과 브랜치가 나뉘기 전의 커밋)
2. main 브랜치의 현재 상태 (HEAD)
3. 병합 대상 브랜치의 현재 상태

이 세 파일들을 참고하면서 수정을 하게 됩니다.

참고로, main 브랜치의 현재 상태가 분기 전의 상태와 같을 때의 merge의 경우 fast-forward merge라고 합니다!

git merge ++



rebase and merge

새로운 브랜치의 시작점을 메인 브랜치 끝으로 옮긴 뒤,
fast-forward merge를 진행합니다!

git rebase main (새로운 브랜치에서)
git merge 브랜치 이름 (main 브랜치에서)

squash and merge

새로운 브랜치에 있던 애들이 main의 새로운 커밋
하나로 이동합니다.

git merge --squash 브랜치이름
git commit -m <커밋 메시지>

(8) branch 삭제

```
● → new_project git:(main) git branch -d login
Deleted branch login (was 965a30d).
● → new_project git:(main) git push origin --delete login
To https://github.com/Eoo0m/new_project.git
- [deleted]          login_
```

git branch -d login: login 브랜치 삭제

git push origin --delete login:
원격 저장소(origin)에 있는 login 브랜치 삭제

작업이 끝난 브랜치를 삭제해줍니다

(9) git restore

```
● → new_project git:(main) ✕ echo "TEMP DEBUG" >> app.py

● → new_project git:(main) ✕ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

no changes added to commit (use "git add" and/or "git commit -a")
● → new_project git:(main) ✕ git restore app.py

● → new_project git:(main) git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working_tree clean
```

git restore --staged app.py:
스테이징 영역에서 파일을 내림(add만 취소)

git restore app.py:
파일 수정 자체를 취소

로컬 작업 공간(Working Directory)에서 발생한 실수를 되돌립니다

(10) git revert

● → `new_project git:(main) git log --oneline -2`

```
7b75b6c (HEAD -> main, origin/main, origin/HEAD) hotfix: add temporary comment
9a9f490 feat: login 기능 추가
```

● → `new_project git:(main) git revert 7b75b6c`
[main f56934b] Revert "hotfix: add temporary comment"
1 file changed, 1 deletion(-)

● → `new_project git:(main) git push`
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 299 bytes | 299.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Eoo0m/new_project.git
7b75b6c..f56934b main -> main

git revert <hash>:

그 커밋을 취소하는 “새 커밋”을 생성

특정 커밋의 내용을 반대로 수행하는 '취소 커밋'을 새로 생성합니다

Git 협업

.gitignore

Git이 추적하지 말아야 할 파일이나 폴더의 목록을 지정

목적:

- 보안 정보 유출 방지: API key, 비밀번호 등
- 불필요한 파일 제외: 캐시(cache) 파일, 로그(log) 파일 등
- 개인 설정 제외: OS에서 생성하는 임시 파일(ex .DS_Store)

작성규칙:

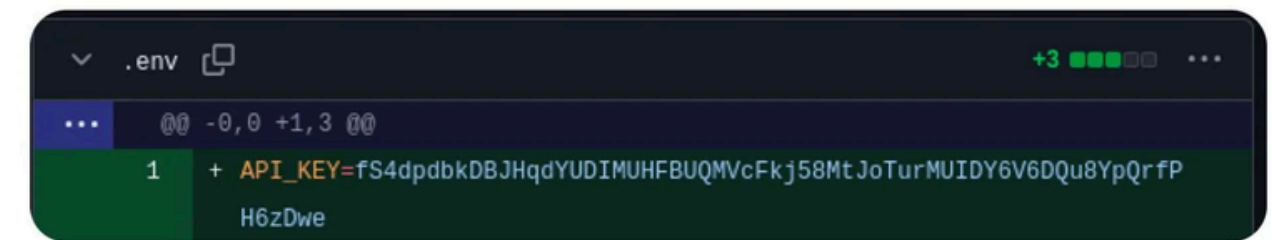
- 파일명 자체를 무시: filename 그대로 입력
- 확장자 전체를 무시: *.ext 입력
- 접두어 조건: prefix* 입력 → 해당 prefix를 가진 파일 전부 무시
- 예외 처리: !filename 이 친구만 예외적으로 추적
- 디렉토리: dirname/ 입력



pokeghost ✓
@pokeghosst

Follow

today's my last day of unpaid internship



12:15 · 29/10/24 · **328K** Views

85

481

8,4K

385



공공예절 - Pull Request

main 브랜치(혹은 dev 브랜치)에 바로 push하는 것은 좋은 습관이 아니에요.
항상 브랜치를 새로 파서, main에 merge 할 때 코드 리뷰어를 꼭 등록합시다!

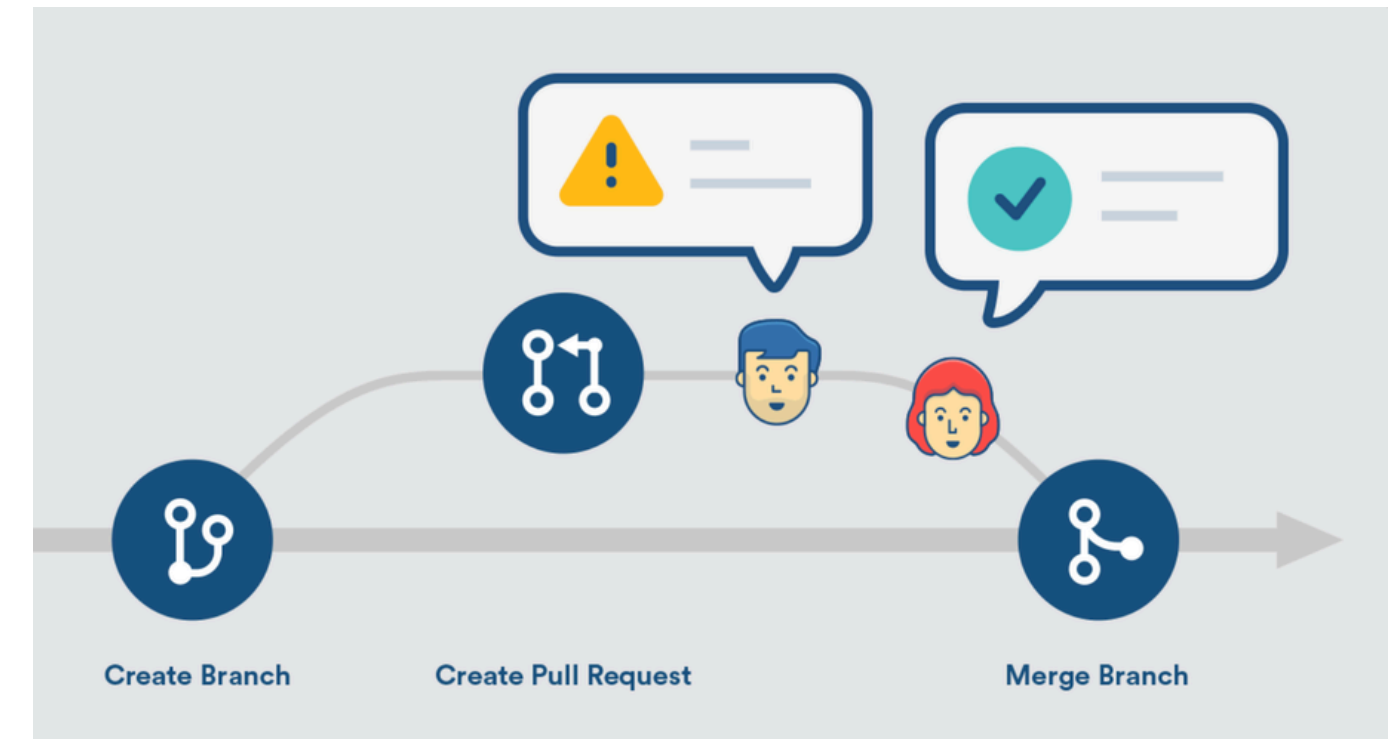
브랜치 이름과 커밋 메시지는 동료들이 보자마자 내용을 알 수 있게 지어야 합니다.

[브랜치 이름 규칙]

- feature/기능명: 새로운 기능을 개발하는 경우
- bugfix/버그이름: 버그 수정
- refactor/기능명: 리팩토링 (기능 변화 X)
- chore/이름: 설정, 빌드, 의존성 등 잡일
- test/이름: 테스트 코드 작성 및 수정

[commit 유형]

- 기능추가(feats): ex) feat: 사용자 로그인 API 구현
- 버그 수정 (fix): ex) fix: 로그인 시 세션 만료 오류 수정
- 스타일 수정 (style): ex) style: 로그인 버튼 레이아웃 수정



공공예절 - Review

Collaboration

feat: Dockerize serving #3

Merged merged 2 commits into `main` from `feat/dockerize-main` last week

Conversation 1 Commits 2 Checks 0 Files changed 2 +35 -0

commented last week

Additional comments

N/A

requested a review from last week

reviewed last week

`docker-compose.yml` Outdated Show resolved

Update `docker-compose.yml` Verified 60ce6e3

merged commit 8938b48 into `main` last week

Pull request successfully merged and closed

You're all set — the `feat/dockerize-main` branch can be safely deleted.

Add a comment

Write Preview

Igtm

Markdown is supported Paste, drop, or click to add files

Comment

Reviewers

Assignees

No one — assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Unsubscribe

You're receiving notifications because your review was requested.

2 participants

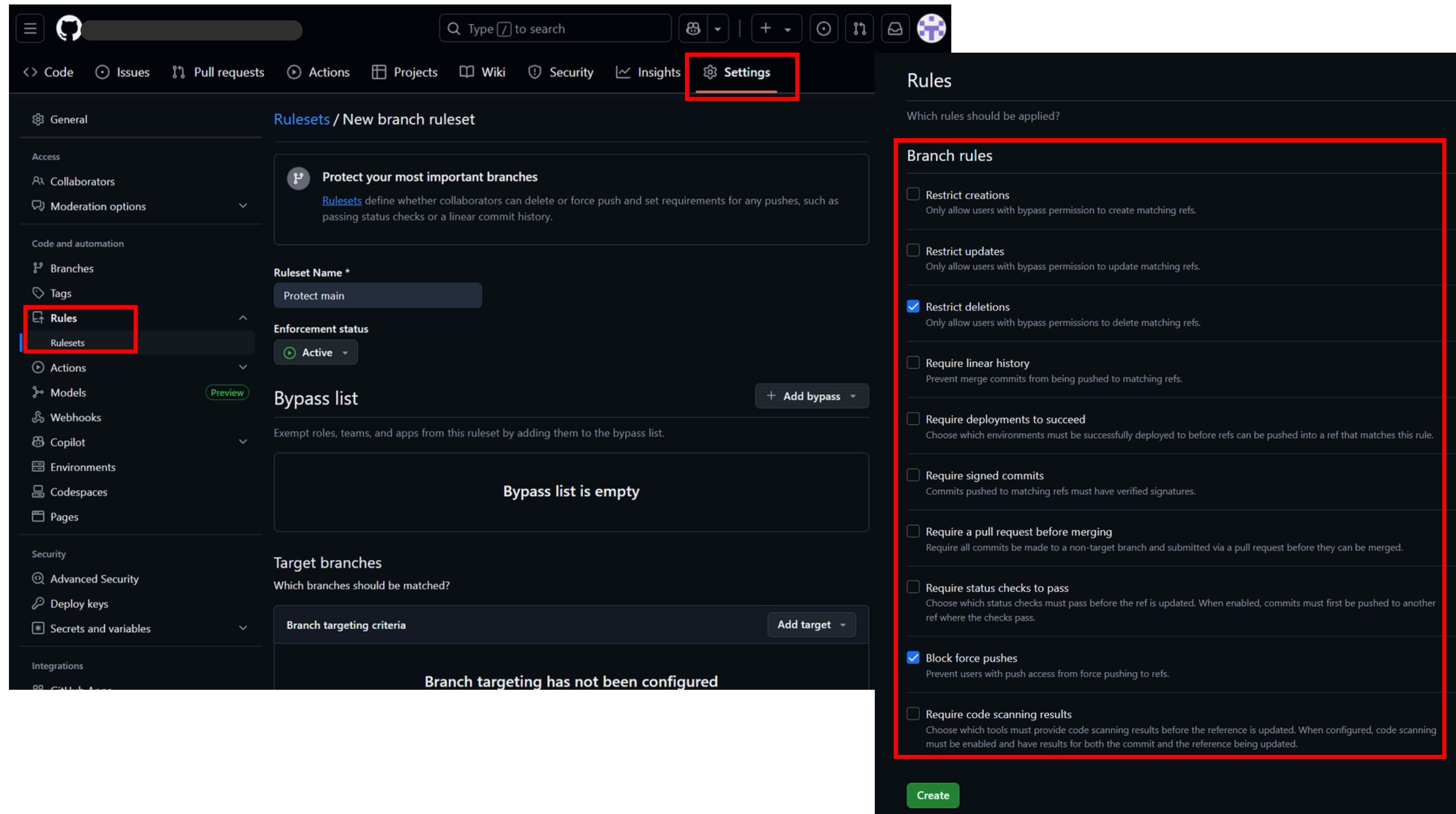
Lock conversation

필요한 경우 코멘트를 남겨주세요!

리뷰어도 코멘트를 남기면 좋아요!

공공예절 - Branch ruleset

Collaboration



The screenshot displays the GitHub 'New branch ruleset' configuration page. The interface is divided into three main sections: a left sidebar, a central configuration area, and a right-hand 'Rules' panel.

Left Sidebar: Contains navigation links for 'General', 'Access', 'Collaborators', 'Moderation options', 'Code and automation', 'Branches', 'Tags', 'Rules' (highlighted with a red box), 'Rulesets', 'Actions', 'Models', 'Webhooks', 'Copilot', 'Environments', 'Codespaces', 'Pages', 'Security', 'Advanced Security', 'Deploy keys', 'Secrets and variables', and 'Integrations'.

Central Configuration Area: Titled 'Rulesets / New branch ruleset', it includes a 'Protect your most important branches' section, a 'Ruleset Name' field (set to 'Protect main'), an 'Enforcement status' dropdown (set to 'Active'), a 'Bypass list' section (currently empty), and a 'Target branches' section (currently showing 'Branch targeting has not been configured').

Right-hand 'Rules' Panel: Titled 'Rules', it lists various rules that can be applied to the branch. The 'Branch rules' section is highlighted with a red box and includes the following rules:

- ☐ Restrict creations: Only allow users with bypass permission to create matching refs.
- ☐ Restrict updates: Only allow users with bypass permission to update matching refs.
- ☒ Restrict deletions: Only allow users with bypass permissions to delete matching refs.
- ☐ Require linear history: Prevent merge commits from being pushed to matching refs.
- ☐ Require deployments to succeed: Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
- ☐ Require signed commits: Commits pushed to matching refs must have verified signatures.
- ☐ Require a pull request before merging: Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
- ☐ Require status checks to pass: Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
- ☒ Block force pushes: Prevent users with push access from force pushing to refs.
- ☐ Require code scanning results: Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.

A 'Create' button is located at the bottom of the 'Rules' panel.

감사합니다!